Sequential RISC-V Processor Design

Batta Venkata Shashank
2024102057

Ganipisetty Nischal
2024102070

Regalla Kowshikh Reddy
2024102053

February 2026

Team: **NRS I-11 Processor**

Introduction

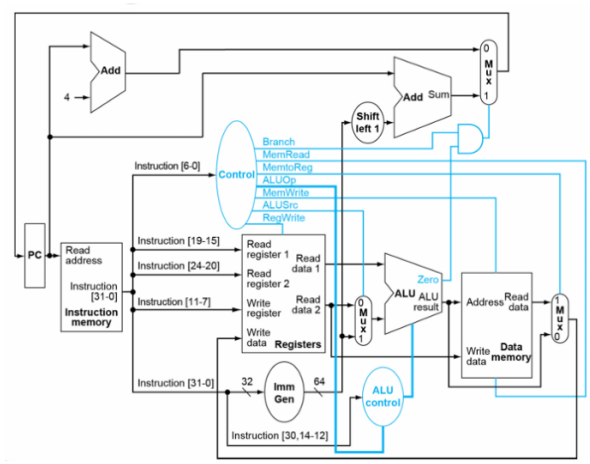

Figure 1: Sequential data path

In this single-cycle RISC-V processor, all components work together to execute one instruction completely in a single clock cycle. The instruction is fetched from memory, decoded, operands are read, the operation is performed by the ALU, memory is accessed if required, and the result is written back to the register file.

The Program Counter (PC) stores the address of the instruction memory. It is updated to the next instruction address using the adders present in the datapath. The first adder is used to fetch the immediate next instruction by adding 4 to the PC value. The second adder is used to jump from one instruction to another during branch operations. This is achieved by adding the shifted immediate value to the PC, where the shift-left operation ensures proper word alignment.

Next, the instruction memory fetches a 32-bit instruction using the address provided by the PC.

This instruction is then sent to three main blocks:

- **Control Unit**: It generates control signals for the ALU, multiplexers, and determines whether load or store operations are to be performed.
- **Register File**: It reads data from the source registers and writes data back to the destination register.
- **Immediate Generation Unit**: It extracts the immediate value from the instruction and performs sign extension.

The ALU performs the required arithmetic or logical operation based on the control signals generated by the ALU control unit.

The data memory is accessed during load and store instructions. Depending on the control signals, it either reads data from memory or writes data to memory.

Aspect	Behavioral Modeling	Structural Modeling
Pros	Easy to write and understand Less code and faster development Good for functional verification Suitable for high-level design	Precise control over hardware structure Closely matches actual hardware Useful for datapath-level design Easier timing analysis
Cons	Limited control over hardware implementation May lead to inefficient hardware Timing details are abstract	Code is lengthy and complex Difficult to modify and scale Time-consuming to design and debug

Table: Comparison between Behavioral and Structural Verilog Modeling

Verilog Preference

The following comparison highlights the trade-offs between behavioral and structural modeling.
From the above table, Behavioral modeling is preferred for fast and simple design, while structural modeling is useful when precise hardware control and detailed datapath representation are required.

1 Program Counter (PC)

The Program Counter (PC) is a special register in the processor that stores the memory address of the next instruction to be executed. It does not store the instruction itself; instead, it holds the address in memory where the instruction is located.

The PC plays a crucial role in controlling the flow of program execution. During each clock cycle, the value stored in the PC is sent to the instruction memory. The instruction memory then uses this address to fetch the corresponding 32-bit instruction.

In a RISC-V processor, each instruction occupies 4 bytes. Therefore, during normal sequential execution, the PC is incremented by 4 after every instruction. This is achieved using an adder in the datapath. The updated value is calculated as:

$$\text{Next PC} = \text{PC} + 4$$

This ensures that the processor moves to the next instruction in memory.

However, in the case of branch or jump instructions, the PC does not simply increment by 4. Instead, a branch target address is calculated by adding a shifted immediate value to the current PC. The shift-left operation ensures proper word alignment. In such cases, the updated PC value becomes:

$$\text{Next PC} = \text{PC} + (\text{Immediate} \ll 1).$$

A multiplexer is used to select between the sequential address (PC + 4) and the branch target address. The selection is controlled by branch control signals generated by the control unit.

The PC is updated at every clock cycle, typically on the positive edge of the clock signal. During system reset, the PC is initialized to a predefined starting address (commonly zero), from where instruction execution begins.

Simulations

```
[Running] PC_tb.v
VCD info: dumpfile Pc_tb.vcd opened for output.
After reset:       pc_out = 0 (expected 0)
PC+4:              pc_out = 4 (expected 4)
PC+4:              pc_out = 8 (expected 8)
PC+4:              pc_out = 12 (expected 12)
Branch to 40:      pc_out = 40 (expected 40)
PC+4 after branch: pc_out = 44 (expected 44)
Mid-run reset:     pc_out = 0 (expected 0)
After reset: pc_out = 4 (expected 4)
PC_tb.v:51: $finish called at 80000 (1ps)
[Done] exit with code=0 in 0.195 seconds
```

Figure 2: Output of PC

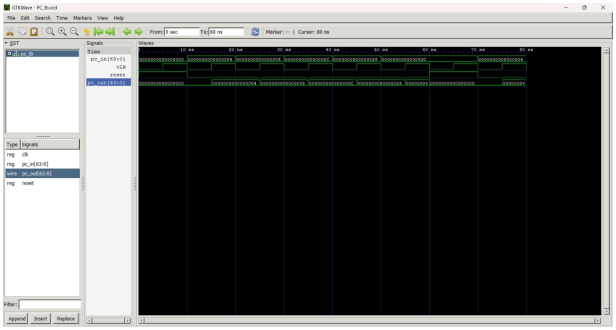

Figure 3: GTK Wave

2 Instruction Memory

Instruction Memory is a memory block in the processor that stores all the program instructions. It contains the machine code that the processor must execute. Unlike data memory, which stores data values, instruction memory stores only instructions.

Each instruction in a RISC-V processor is 32 bits (4 bytes) long. The instruction memory is organized as a collection of memory locations, where each location has a unique address. These addresses are used by the processor to access specific instructions.

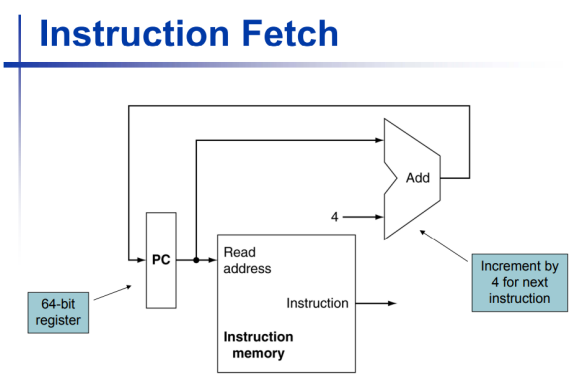

Figure 4: Instruction fetch

The Program Counter (PC) provides the address to the instruction memory. When the processor begins execution, the PC holds the address of the first instruction. This address is sent to the instruction memory, which then outputs the corresponding 32-bit instruction.

The process of obtaining an instruction from instruction memory is called **instruction fetch**. This is the first stage of instruction execution.

In normal sequential execution, the PC increments by 4 after every instruction. Since each instruction occupies 4 bytes, this ensures that the processor fetches the next instruction in memory.

Instruction memory is typically read-only during program execution. This means that instructions are not modified while the program is running. In hardware implementations, instruction memory may be implemented using ROM (Read Only Memory) or initialized memory blocks.

In a single-cycle RISC-V datapath, instruction memory has:

- One input: Address (from the PC)
- One output: 32-bit Instruction

When the address changes, the instruction memory outputs the instruction stored at that address.

In the single-cycle RISC-V datapath, instruction memory receives the address from the PC and provides the instruction to the control unit, register file, and immediate generation block for further processing.

Instruction Formats

R-Type Instruction

An R-type instruction is 32 bits and is divided as follows:

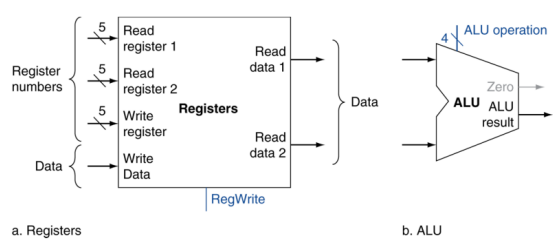

Figure 5: R-format

funct7	rs2	rs1	funct3	rd	opcode
31–25	24–20	19–15	14–12	11–7	6–0

As shown in Figure 1,

- **opcode (6–0)** → Control Unit
- **rs1 (19–15)** → Register File (Read Register 1)
- **rs2 (24–20)** → Register File (Read Register 2)
- **rd (11–7)** → Register File (Write Register)
- **funct3 (14–12)** → ALU Control
- **funct7 (31–25)** → ALU Control

For R-type instructions, only the Register File and ALU are used. Immediate Generator and Data Memory are not used.

I-Type Instruction Bit Division

An I-type instruction is 32 bits and is divided as follows:

Immediate	rs1	funct3	rd	opcode
31–20	19–15	14–12	11–7	6–0

As shown in Figure 1,

- **opcode (6–0)** → Control Unit
- **rs1 (19–15)** → Register File (Read Register 1)
- **rd (11–7)** → Register File (Write Register)
- **funct3 (14–12)** → ALU Control
- **Immediate (31–20)** → Immediate Generator

For I-type instructions, the second ALU operand comes from the Immediate Generator. Data Memory is used only in load instructions.

B-Type Instruction Bit Division

A B-type instruction is 32 bits and is divided as follows:

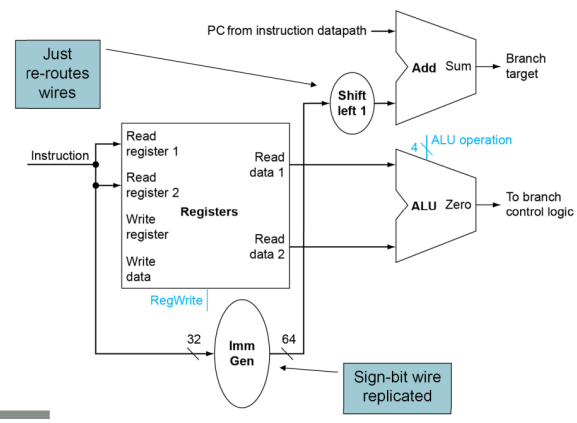

Figure 6: B-Type

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
31	30–25	24–20	19–15	14–12	11–8	7	6–0

As shown in Figure 1,

- **opcode (6–0)** → Control Unit
- **rs1 (19–15)** → Register File (Read Register 1)
- **rs2 (24–20)** → Register File (Read Register 2)
- **funct3 (14–12)** → ALU Control
- **Immediate bits** → Immediate Generator

For B-type instructions, the Immediate Generator forms the branch offset. The ALU compares the two register values, and the Branch control signal determines whether the PC is updated to the branch target address.

S-Type Instruction Bit Division

An S-type instruction is 32 bits and is divided as follows:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
31–25	24–20	19–15	14–12	11–7	6–0

As shown in Figure 1,

- **opcode (6–0)** → Control Unit
- **rs1 (19–15)** → Register File (Read Register 1)
- **rs2 (24–20)** → Register File (Read Register 2)
- **funct3 (14–12)** → ALU Control
- **Immediate bits (31–25 and 11–7)** → Immediate Generator

For S-type instructions, the Immediate Generator combines the two immediate fields to form the store offset. The ALU calculates the memory address using rs1 and the immediate value. The value from rs2 is written into Data Memory.

Simulations

```
VCD info: dumpfile Instruction_mem_tb.vcd opened for output.
addr 0   | GOT = 00500113 | EXP = 00500113
addr 4   | GOT = 00a00193 | EXP = 00A00193
addr 8   | GOT = 003100b3 | EXP = 003100B3
addr 32  | GOT = 0012b023 | EXP = 0012B023
addr 56  | GOT = 00a086b3 | EXP = 00A086B3
addr 100 | GOT = 00000000 | EXP = 00000000
after reset addr 0 | GOT = 00500113 | EXP = 00500113
Instruction_mem_tb.v:51: $finish called at 86000 (1ps)
[Done] exit with code=0 in 0.217 seconds
```

Figure 7: Instruction memory output

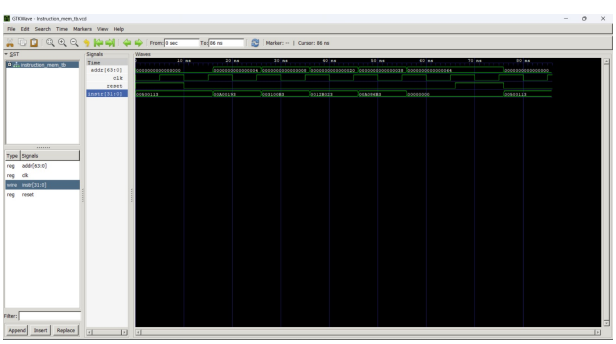

Figure 8: GTK Wave

3 Register File

The Register File is a small, high-speed memory unit inside the processor that stores temporary data required during instruction execution. It contains a set of registers that hold operands, intermediate results, and final computation results.

In a RISC-V (RV64I) processor, the register file consists of 32 general-purpose registers, each 64 bits wide. These registers are named x0 to x31. Among them, register x0 is hardwired to zero, meaning it always contains the value 0 and cannot be modified.

The register file is designed to support multiple operations simultaneously. It typically has:

- Two read ports
- One write port

The two read ports allow the processor to read two register values at the same time, which is necessary for arithmetic and logical instructions such as addition and subtraction. The write port allows the processor to store the result of an operation back into a destination register.

During instruction execution, the instruction provides:

- `rs1` – address of the first source register
- `rs2` – address of the second source register
- `rd` – address of the destination register

The register file reads the values stored in `rs1` and `rs2` and sends them to the Arithmetic Logic Unit (ALU) for processing. After the ALU completes the operation, the result is written back into the register specified by `rd`, provided the `RegWrite` control signal is enabled.

Writing to the register file typically occurs on the positive edge of the clock signal. The read operation is usually faster and may be asynchronous in simple single-cycle designs.

In the single-cycle RISC-V datapath, the register file acts as the main source of operands for the ALU and stores the final computation results for future instructions. Its high speed and efficient design make it a critical component of processor performance.

Simulations

```
[Running] RegMem_tb.v
VCD info: dumpfile RegMem_tb.vcd opened for output.
RESET TEST: x1 = 0000000000000000 (expected 0)
WRITE TEST: x1 = deadbeefcafebabe (expected DEAD_BEEF_CAFE_BABE)
WRITE TEST: x0 = 0000000000000000 (expected 0)
x0 TEST: x0 = 0000000000000000 (expected 0)
MULTI WRITE: x2 =                  10 (expected 10)
MULTI WRITE: x3 =                  20 (expected 20)
DUAL READ: x3 =                   20 ,                  20 (expected 20,20)
RESET AFTER WRITE: x1=0000000000000000 x2=0000000000000000 (expected 0,0)
POST RESET WRITE: x5=123456789abcdef0 (expected 123456789ABCDEF0)
register_file.txt dumped (cycle_count = 42)
RegMem_tb.v:94: $finish called at 106000 (1ps)
[Done] exit with code=0 in 0.174 seconds
```

Figure 9: Expected Register Memory

```
register_file.txt
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
123456789abcdef0
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
42
```

Figure 10: Register File output

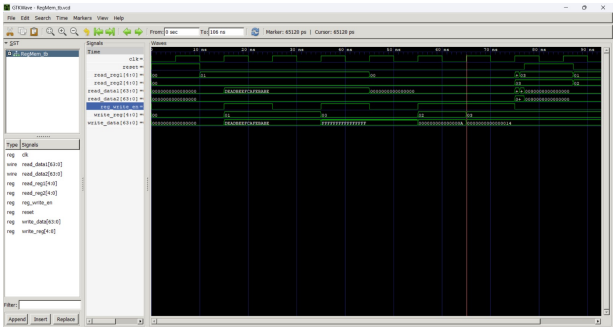

Figure 11: GTK Wave

4 Control Unit

The Control Unit is responsible for generating control signals that coordinate the operation of all components in the datapath. It does not execute arithmetic or memory operations directly; instead, it orchestrates datapath behavior by generating appropriate control signals.

The Control Unit receives the `opcode` field (bits 6–0) from the instruction. Based on the opcode, it identifies the instruction type (R-type, I-type, S-type, B-type, etc.) and generates the appropriate control signals.

The main control signals generated in the sequential RISC-V datapath are:

- **RegWrite** – Enables writing into the Register File.
- **ALUSrc** – Selects the second ALU operand (register or immediate).
- **MemRead** – Enables reading from Data Memory.
- **MemWrite** – Enables writing to Data Memory.
- **MemtoReg** – Selects whether the value written back to the register file comes from the ALU or Data Memory.
- **Branch** – Enables branch decision logic for branch instructions.
- **ALUOp** – Provides operation information to the ALU Control unit.

Based on the instruction type, these control signals are set accordingly. For example, R-type instructions enable `RegWrite` and use register operands, while load instructions enable `MemRead` and select memory data for write-back.

In the sequential datapath, the Control Unit plays a central role by decoding the instruction and generating signals that determine how each hardware block operates during execution.

Simulations

```
[Running] Control_unit_tb.v
VCD info: dumpfile Control_unit_tb.vcd opened for output.
R-TYPE   GOT | Br=0 MR=0 M2R=0 ALUOp=10 MW=0 ASrc=0 RW=1
R-TYPE   EXP | Br=0 MR=0 M2R=0 ALUOp=10 MW=0 ASrc=0 RW=1

I-TYPE   GOT | Br=0 MR=0 M2R=0 ALUOp=11 MW=0 ASrc=1 RW=1
I-TYPE   EXP | Br=0 MR=0 M2R=0 ALUOp=11 MW=0 ASrc=1 RW=1

LOAD     GOT | Br=0 MR=1 M2R=1 ALUOp=00 MW=0 ASrc=1 RW=1
LOAD     EXP | Br=0 MR=1 M2R=1 ALUOp=00 MW=0 ASrc=1 RW=1

STORE    GOT | Br=0 MR=0 M2R=0 ALUOp=00 MW=1 ASrc=1 RW=0
STORE    EXP | Br=0 MR=0 M2R=0 ALUOp=00 MW=1 ASrc=1 RW=0

BRANCH   GOT | Br=1 MR=0 M2R=0 ALUOp=01 MW=0 ASrc=0 RW=0
BRANCH   EXP | Br=1 MR=0 M2R=0 ALUOp=01 MW=0 ASrc=0 RW=0

INVALID  GOT | Br=0 MR=0 M2R=0 ALUOp=00 MW=0 ASrc=0 RW=0
INVALID  EXP | Br=0 MR=0 M2R=0 ALUOp=00 MW=0 ASrc=0 RW=0

Control_unit_tb.v:47: $finish called at 6000 (1ps)
[Done] exit with code=0 in 0.147 seconds
```

Figure 12: Control Unit output

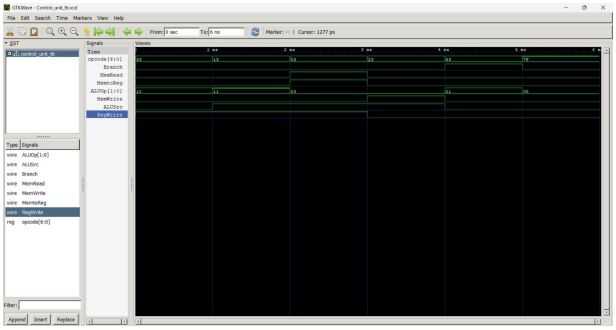

Figure 13: GTK Wave

5 Immediate Generation

The Immediate Generation (ImmGen) unit is responsible for extracting the immediate value from the instruction and converting it into a properly formatted 64-bit value in the RV64I architecture. Many RISC-V instructions use an immediate constant instead of a second register operand, and this unit prepares that value for further processing.

The Immediate Generation unit receives the complete 32-bit instruction as input. Based on the instruction format identified by the opcode, it selects the appropriate bits that represent the immediate value. The extracted immediate is then sign-extended to 64 bits before being used in the datapath.

Immediate Formation in Different Instruction Types

- **I-Type:** Immediate is taken from bits 31–20 and sign-extended to 64 bits. Used in arithmetic-immediate and load instructions.

- **S-Type:** Immediate is formed by combining bits 31–25 and 11–7, then sign-extended. Used in store instructions.

- **B-Type:** Immediate bits are taken from positions 31, 30–25, 11–8, and 7. These bits are rearranged, shifted left by one, and sign-extended. Used in branch instructions.

In the sequential datapath, the output of the Immediate Generation unit is connected to the ALU input through a multiplexer. When the `ALUSrc` control signal is enabled, the ALU uses the immediate value instead of the second register operand. For branch and jump instructions, the immediate value is also used in calculating the target address.

In summary, the Immediate Generation unit extracts, arranges, and sign-extends the immediate field from the instruction and provides it to the ALU or branch logic for further computation.

Simulations

```
[Running] Immediate_gen_tb.v
VCD info: dumpfile Immediate_gen_tb.vcd opened for output.
I-type max  (+2047) = 2047
I-type min  (-2048) = -2048
S-type max  (+2047) = 2047
S-type min  (-2048) = -2048
B-type min  (+1)    = 1
B-type min  (-1)    = -1
B-type max  (+2047) = 2047
B-type min  (-2048) = -2048
R-type imm          = 0
Immediate_gen_tb.v:35: $finish called at 9000 (1ps)
[Done] exit with code=0 in 0.165 seconds
```

Figure 14: Immediate Generation output

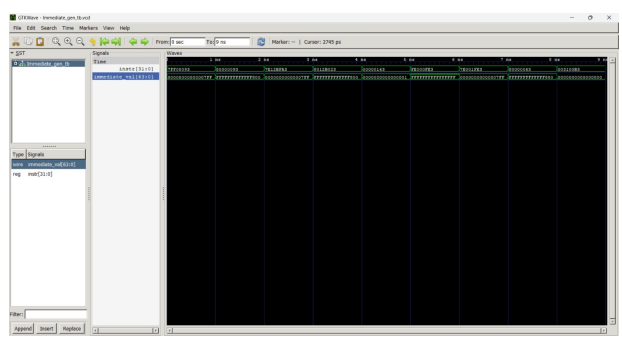

Figure 15: GTK Wave

6 ALU Control

The ALU Control unit generates the specific control signals required by the Arithmetic Logic Unit (ALU). It receives a 2-bit `ALUOp` signal from the main Control Unit and additional function fields from the instruction. Using combinational logic, it determines the exact ALU operation to be performed.

Inputs to ALU Control

- **ALUOp (2 bits)** – Derived from the opcode by the main Control Unit.
- **Function field** – Derived from the instruction fields (such as funct3 and funct7).

Operation Based on ALUOp

- **ALUOp = 00:** Used for load (`ld`) and store (`sd`) instructions. The ALU performs addition to compute the effective memory address.
- **ALUOp = 01:** Used for branch equal (`beq`) instruction. The ALU performs subtraction to compare the two register values.
- **ALUOp = 10:** Used for R-type instructions. The exact ALU operation is determined using the function field of the instruction.

ALU Control Encoding

The ALU Control unit generates a 4-bit control signal for the ALU. The encoding is as follows:

ALU Control	Function
0000	AND
0001	OR
0010	ADD
0110	SUBTRACT

Example Operation Mapping

- For `ld` and `sd`: ALUOp = 00 → ALU Control = 0010 (ADD)
- For `beq`: ALUOp = 01 → ALU Control = 0110 (SUBTRACT)
- For R-type instructions (ALUOp = 10):
  - ADD → 0010
  - SUB → 0110
  - AND → 0000
  - OR → 0001

Simulations

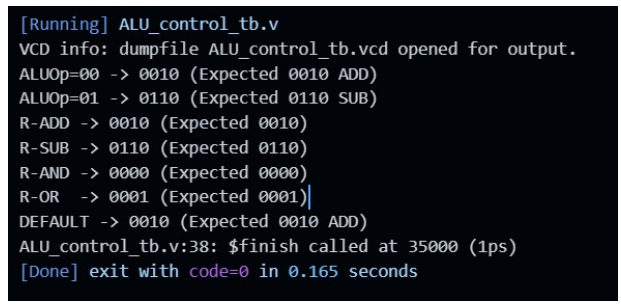

Figure 16: ALU Control output

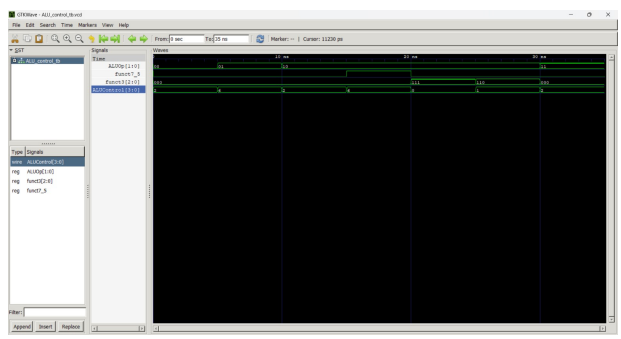

Figure 17: GTK Wave

7 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the main computation block of the processor. It performs arithmetic

and logical operations required during instruction execution. Almost every instruction in the RISC-V processor uses the ALU in some form.

The ALU operates on two input operands and produces a result along with a status output known as the Zero flag.

Inputs to the ALU

The ALU receives the following inputs:

- First operand from the Register File (`Read Data 1`)
- Second operand selected by the `ALUSrc` multiplexer
  - If `ALUSrc = 0`, the operand comes from the Register File (`Read Data 2`)
  - If `ALUSrc = 1`, the operand comes from the Immediate Generator
- A 4-bit ALU control signal from the ALU Control unit

ALU Operations

Based on the 4-bit ALU control signal, the ALU performs operations such as:

- `0000` – AND
- `0001` – OR
- `0010` – ADD
- `0110` – SUBTRACT

The 4-bit control encoding ensures modular ALU design and simplifies operation decoding.

Zero Flag

The ALU generates a Zero output signal. If the result of the operation is zero, the Zero flag is set to 1; otherwise, it is set to 0. This signal is used in branch instructions such as `beq` to determine whether the branch condition is satisfied.

Role in Different Instructions

- **R-Type:** Performs arithmetic or logical operations between two registers.
- **I-Type:** Performs operations between a register and an immediate value.
- **Load/Store:** Computes the effective memory address.
- **Branch:** Compares register values using subtraction.

The ALU connects the Register File, Immediate Generator, Data Memory, and branch logic. It plays a central role in executing arithmetic, logical, and address calculation operations.

Simulations

```
[Running] ALU_tb.v
VCD info: dumpfile ALU_tb.vcd opened for output.
PASS | op=0010 A=0000000000000005 B=0000000000000003 => 0000000000000008 Z=0
PASS | op=0010 A=ffffffffffffffff B=0000000000000001 => 0000000000000000 Z=1
PASS | op=0010 A=7fffffffffffffff B=0000000000000001 => 8000000000000000 Z=0
PASS | op=0010 A=aaaaaaaaaaaaaaaa B=5555555555555555 => ffffffffffffffff Z=0
PASS | op=0110 A=000000000000000a B=0000000000000004 => 0000000000000006 Z=0
PASS | op=0110 A=0000000000000001 B=0000000000000001 => 0000000000000000 Z=1
PASS | op=0110 A=0000000000000000 B=0000000000000001 => ffffffffffffffff Z=0
PASS | op=0110 A=8000000000000000 B=0000000000000001 => 7fffffffffffffff Z=0
PASS | op=0000 A=ffffffffffffffff B=0000000000000000 => 0000000000000000 Z=1
PASS | op=0000 A=aaaaaaaaaaaaaaaa B=5555555555555555 => 0000000000000000 Z=1
PASS | op=0000 A=deadbeefcafebabe B=ffffffff00000000 => deadbeef00000000 Z=0
PASS | op=0001 A=0000000000000000 B=ffffffffffffffff => ffffffffffffffff Z=0
PASS | op=0001 A=aaaaaaaaaaaaaaaa B=5555555555555555 => ffffffffffffffff Z=0
PASS | op=0001 A=00000000ffffffff B=ffffffff00000000 => ffffffffffffffff Z=0
PASS | op=1111 A=0000000000001234 B=0000000000005678 => 0000000000000000 Z=1
ALU_tb.v:71: $finish called at 75000 (1ps)
[Done] exit with code=0 in 0.414 seconds
```

Figure 18: ALU Output

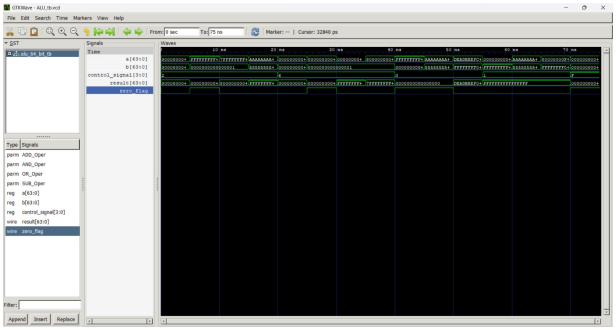

Figure 19: GTK Wave

8 Data Memory

Data Memory stores and retrieves program data during load and store operations. Unlike Instruction Memory, which stores program instructions, Data Memory stores data values such as variables and array elements.

In the sequential RISC-V datapath, Data Memory is accessed only during load and store instructions. It is not used for R-type or branch instructions.

Inputs to Data Memory

The Data Memory receives the following inputs:

- **Address:** The effective memory address calculated by the ALU.
- **Write Data:** Data from the Register File (used in store instructions).
- **MemRead:** Control signal that enables reading from memory.
- **MemWrite:** Control signal that enables writing to memory.

Load Operation

In load instructions (e.g., `ld`), the ALU computes the effective address by adding the base register value and the immediate offset. When `MemRead` is enabled, the Data Memory reads the value stored at that address. The output data is then selected by the `MemtoReg` multiplexer and written back to the Register File.

Store Operation

In store instructions (e.g., `sd`), the ALU computes the effective memory address. When `MemWrite` is enabled, the value from the Register File is written into the memory location specified by the calculated address.

Output of Data Memory

The Data Memory produces a read data output during load operations. This output is routed through a multiplexer to the Register File for write-back.

Simulations

```
[Running] DataMem_tb.v
VCD info: dumpfile datamem_tb.vcd opened for output.
PASS | addr=0 data=0000000000000000
PASS | addr=100 data=0000000000000000
PASS | addr=0 data=deadbeefcafebabe
PASS | addr=16 data=0102030405060708
PASS | addr=17 data=0203040506070800
PASS | addr=50 data=5555555555555555
PASS | addr=1016 data=cafebabedeadbeef
PASS | addr=1017 data=0000000000000000
PASS | addr=0 data=0000000000000000
PASS | addr=1016 data=0000000000000000
PASS | addr=200 data=0000000000000000
PASS | addr=10 data=000000000000000f
PASS | addr=34 data=fffffffffffffffb
PASS | addr=300 data=0000000000000000
DataMem_tb.v:121: $finish called at 147000 (1ps)
[Done] exit with code=0 in 0.196 seconds
```

Figure 20: Data Memory Output

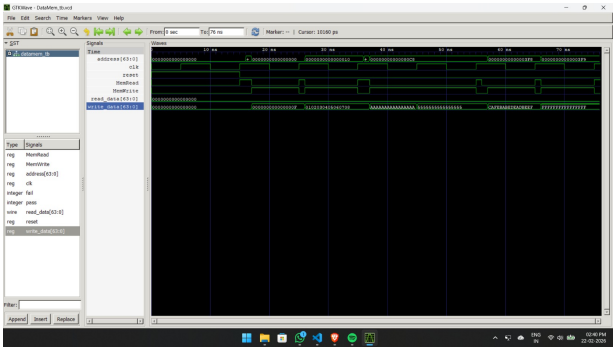

Figure 21: GTK Wave

9 Multiplexers

Multiplexers (MUXes) are combinational logic components used to select one input from multiple inputs based on a control signal. In the sequential RISC-V datapath, multiplexers control the flow of data between different hardware blocks.

Several multiplexers are used in the datapath, each serving a specific purpose.

1. ALU Source Multiplexer (ALUSrc)

This multiplexer selects the second input to the ALU.

- If `ALUSrc = 0`, the second operand comes from the Register File (rs2).
- If `ALUSrc = 1`, the second operand comes from the Immediate Generator.

This multiplexer is essential for distinguishing between R-type instructions (register operands) and I-type, load, or store instructions (immediate operand).

2. MemtoReg Multiplexer

This multiplexer selects the data that is written back to the Register File.

- If `MemtoReg = 0`, the ALU result is written to the register.
- If `MemtoReg = 1`, the data read from Data Memory is written to the register.

It is mainly used in load instructions.

3. PC Source Multiplexer (Branch MUX)

This multiplexer selects the next value of the Program Counter (PC).

- If the branch condition is false, the next PC is `PC + 4`.
- If the branch condition is true, the next PC is the branch target address.

The selection is controlled by the `Branch` signal combined with the Zero output of the ALU.

Multiplexers play a critical role in directing data flow within the processor. They allow the datapath to support different instruction types by selecting appropriate inputs based on control signals generated by the Control Unit.

Simulations

```
[Running] Multiplexer_tb.v
VCD info: dumpfile multiplexer_tb.vcd opened for output.
PASS | S=0 I1=00000000000000aa I2=00000000000000bb -> out=00000000000000aa
PASS | S=1 I1=00000000000000aa I2=00000000000000bb -> out=00000000000000bb
PASS | S=0 I1=0000000000000000 I2=0000000000000000 -> out=0000000000000000
PASS | S=1 I1=0000000000000000 I2=0000000000000000 -> out=0000000000000000
PASS | S=0 I1=ffffffffffffffff I2=ffffffffffffffff -> out=ffffffffffffffff
PASS | S=1 I1=ffffffffffffffff I2=ffffffffffffffff -> out=ffffffffffffffff
PASS | S=0 I1=aaaaaaaaaaaaaaaa I2=5555555555555555 -> out=aaaaaaaaaaaaaaaa
PASS | S=1 I1=aaaaaaaaaaaaaaaa I2=5555555555555555 -> out=5555555555555555
PASS | S=1 I1=ffffffffffffffff I2=0000000000000000 -> out=0000000000000000
PASS | S=0 I1=ffffffffffffffff I2=0000000000000000 -> out=ffffffffffffffff
Multiplexer_tb.v:54: $finish called at 10000 (1ps)
[Done] exit with code=0 in 0.179 seconds
```

Figure 22: Multiplexer output

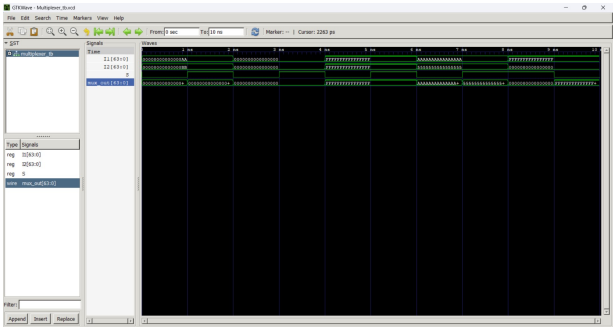

Figure 23: GTK Wave

10 Adder Blocks

Adder blocks are combinational circuits used to perform addition operations in the datapath. In the sequential RISC-V processor, adders are primarily used for updating the Program Counter (PC) and calculating branch target addresses.

1. PC + 4 Adder

The first adder is used to increment the Program Counter by 4. Since each instruction in RISC-V is 4 bytes (32 bits) long, the next sequential instruction is located at:

$$\text{Next PC} = \text{PC} + 4$$

This adder ensures that during normal sequential execution, the processor moves to the next instruction in memory.

2. Branch Target Adder

The second adder is used to compute the branch target address. For branch instructions, the Immediate Generation unit produces a signed offset value. This value is shifted left by one bit (to maintain proper alignment) and added to the current PC:

$$\text{Branch Target} = \text{PC} + (\text{Immediate} \ll 1)$$

This calculated address becomes the next PC value if the branch condition is satisfied.

Role in the Datapath

Both adders operate in parallel:

- The PC + 4 adder computes the sequential address.
- The branch adder computes the potential branch address.

A multiplexer selects between these two outputs based on the Branch control signal and the Zero flag from the ALU.

Simulations

```
[Running] Adder_branch_tb.v
VCD info: dumpfile Adder_branch_tb.vcd opened for output.
PASS | r1=00000000000003e8 r2=0000000000000020 -> rd=0000000000000408
PASS | r1=00000000000003e8 r2=fffffffffffffff0 -> rd=00000000000003d8
PASS | r1=0000000040000000 r2=0000000000000000 -> rd=0000000040000000
PASS | r1=ffffffffffffffff r2=0000000000000001 -> rd=0000000000000000
PASS | r1=aaaaaaaaaaaaaaaa r2=5555555555555555 -> rd=ffffffffffffffff
PASS | r1=aaaaaaaaaaaaaaaa r2=5555555555555556 -> rd=0000000000000000
PASS | r1=0000000000000004 r2=ffffffffffffffe0 -> rd=ffffffffffffffe4
Adder_branch_tb.v:43: $finish called at 140000 (1ps)
[Done] exit with code=0 in 0.25 seconds
```

Figure 24: Adder Block output

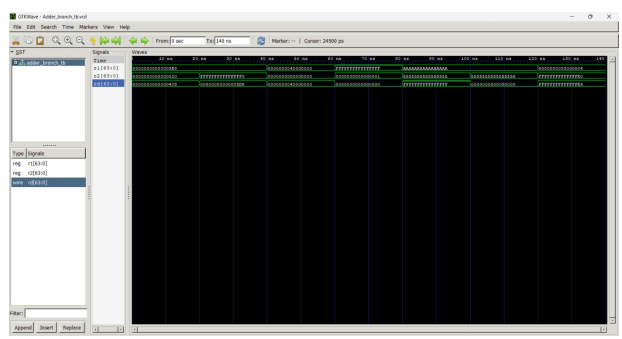

Figure 25: GTK Wave

11 Combinational Circuit

Simulations

```
VCD info: dumpfile seq_tb.vcd opened for output.
Team: 'NRS i11 Processor' RISC-V Sequential Processor Simulation
Reseted,Starting Program
Cycle 1: PC=0, Instr=0x00500113
Cycle 2: PC=4, Instr=0x00a00193
Cycle 3: PC=8, Instr=0x003100b3
Cycle 4: PC=12, Instr=0x40310133
Cycle 5: PC=16, Instr=0x0031f233
Cycle 6: PC=20, Instr=0x0041f2b3
Cycle 7: PC=24, Instr=0x00416333
Cycle 8: PC=28, Instr=0x003163b3
Cycle 9: PC=32, Instr=0x0012b023
Cycle 10: PC=36, Instr=0x0002b503
Cycle 11: PC=40, Instr=0x0062bc23
Cycle 12: PC=44, Instr=0x0182b583
Cycle 13: PC=48, Instr=0x00520463
Cycle 14: PC=56, Instr=0x00a086b3
Cycle 15: PC=60, Instr=0x00000000
All Instruction are implimented,No of Cycles: 15
seq_tb.v:20: $finish called at 165000 (1ps)
[Done] exit with code=0 in 0.753 seconds
```

Figure 26: Combined sequential output

```
register_file.txt
0000000000000000
000000000000000f
fffffffffffffffb
000000000000000a
000000000000000a
000000000000000a
fffffffffffffffb
fffffffffffffffb
0000000000000000
0000000000000000
000000000000000f
fffffffffffffffb
0000000000000000
000000000000001e
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
15
```

Figure 27: Register file output for given Test case

Fibonacci

instructions_exp.txt

```
addi x1, x0, 1          # x1 = 1  (fib_prev = fib[1])
addi x2, x0, 1          # x2 = 1  (fib_curr = fib[2])
addi x4, x0, 0          # x4 = 0  (base memory address)
addi x5, x0, 8          # x5 = 8  (loop counter: 8 more terms after first 2)
addi x6, x0, 1          # x6 = 1  (constant 1 for counter)
addi x8, x0, 0          # x8 = 0  (current memory offset)
sd   x1, 0(x4)          # mem[0]  = 1  (fib[1])
sd   x2, 8(x4)          # mem[8]  = 1  (fib[2])
addi x8, x0, 16         # x8 = 16 (next store offset = mem[16])
add  x3, x1, x2         # x3 = fib_prev + fib_curr = next fib
add  x9, x4, x8         # x9 = base + offset (store address)
sd   x3, 0(x9)          # mem[x9] = x3  (store fib[n])
or   x1, x2, x0         # x1 = x2 | 0 = x2  (fib_prev = fib_curr)
or   x2, x3, x0         # x2 = x3 | 0 = x3  (fib_curr = fib_next)
addi x8, x8, 8          # x8 = x8 + 8  (advance offset)
sub  x5, x5, x6         # x5 = x5 - 1
beq  x5, x0, 8          # if counter==0: exit loop (skip back-branch)
beq  x0, x0, -32        # unconditional: jump back to loop start

ld   x10, 0(x4)         # x10 = mem[0]  = fib[1]  = 1
ld   x11, 8(x4)         # x11 = mem[8]  = fib[2]  = 1
ld   x12, 16(x4)        # x12 = mem[16] = fib[3]  = 2
ld   x13, 24(x4)        # x13 = mem[24] = fib[4]  = 3
ld   x14, 32(x4)        # x14 = mem[32] = fib[5]  = 5
ld   x15, 40(x4)        # x15 = mem[40] = fib[6]  = 8
ld   x16, 48(x4)        # x16 = mem[48] = fib[7]  = 13
ld   x17, 56(x4)        # x17 = mem[56] = fib[8]  = 21
ld   x18, 64(x4)        # x18 = mem[64] = fib[9]  = 34
ld   x19, 72(x4)        # x19 = mem[72] = fib[10] = 55

add  x20, x18, x19      # x20 = 34+55 = 89 = fib[11]
beq  x10, x11, 8        # taken: fib[1]==fib[2]==1, skip next
addi x21, x0, 999       # SKIPPED (x21 stays 0)
and  x22, x10, x11      # x22 = 1 & 1 = 1  (both fib[1]=fib[2]=1)
sub  x23, x19, x18      # x23 = 55-34 = 21 = fib[8] (Fib property check)
```

Figure 28: INPUT

register_file.txt

```
0000000000000000
0000000000000022
0000000000000037
0000000000000037
0000000000000000
0000000000000000
0000000000000001
0000000000000000
0000000000000050
0000000000000048
0000000000000001
0000000000000001
0000000000000002
0000000000000003
0000000000000005
0000000000000008
000000000000000d
0000000000000015
0000000000000022
0000000000000037
0000000000000059
0000000000000000
0000000000000001
0000000000000015
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
95
```

Figure 29: Output of Register File

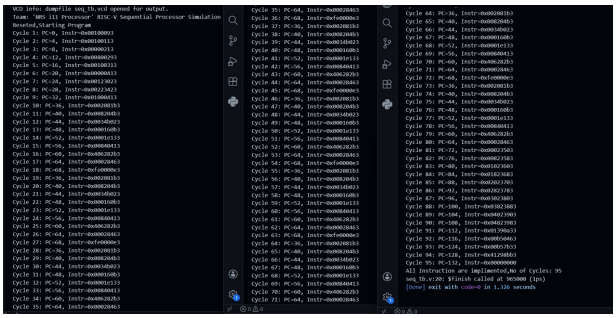

Figure 30: Output for 10 numbers

12 Conclusion

This project successfully implements a single-cycle RV64I RISC-V processor using structural Verilog modeling. All core modules, including the Program Counter, Register File, Arithmetic Logic Unit (ALU), Control Unit, Immediate Generation Unit, and Data Memory, were designed and verified through simulation.

The processor correctly executes arithmetic, memory, and branch instructions, as demonstrated by the Fibonacci test case. The simulation results confirm proper datapath functionality and accurate control signal generation.

Although the single-cycle architecture simplifies control logic and provides conceptual clarity, it suffers from performance limitations due to long critical path delays. Since all instructions are executed within a single clock cycle, the clock period must accommodate the slowest instruction, reducing overall efficiency.

Future work may involve extending this design to a pipelined architecture to improve throughput and performance. Additional enhancements could include hazard detection mechanisms, forwarding units, and support for a broader subset of the RISC-V instruction set.